Use Migrations to Create Database Tables

We have two schemas in our app, but we don't have corresponding database tables yet. Luckily for us, Ecto includes migration support out of the box. This will feel similar to ActiveRecord migrations, so you'll quickly be comfortable if you've used those before.

Migrations are scripts that modify your database. They are often used to add tables, add or modify columns, add indices, and so forth. You can even use migrations to update the data in your database, but this is an advanced case that we won't cover in this book.

Let's look at the basics of migrations and then define migrations for our two schemas.

Migration Basics

`Ecto.Migration` is implemented in a child library of Ecto called ecto_sql. This library contains all of the SQL-specific aspects of Ecto because Ecto isn't tied to a single database technology.

We'll cover a few of the key migration functions, but the migration documentation[13] goes into great detail for all available functions. The "Phoenix: Ecto Migrations Cheatsheet"[14] is useful as it covers all of the main patterns you'll use in migrations.

Often, multiple migration functions will be combined together. For example, the code `create table(:widgets) do / end` uses the `create/2` and `table/1` functions to create a new table, and `create index(:widgets, [:name])` uses `create/1` with `index/2` to create a new index. The most commonly used functions are:

create table(:name) do / end
: Creates a database table with the fields added in the `do` block.

add :field_name, :field_type
: Adds a field to the database table. This can only be used inside of a create table block.

modify :field_name, :field_type
: Edits an existing field, such as changing the type or nullability of the field. This can only be used inside of an alter table block.

13. https://hexdocs.pm/ecto_sql/Ecto.Migration.html
14. https://devhints.io/phoenix-migrations